

Zeon Systems Take Home : Task 1

Tube Detection and Orientation

Rishi Divyakirti

May 16, 2026

Contents

1	Introduction	3
2	Methodology	3
2.1	Stage 1: Coarse Detection (YOLOv8 OBB)	3
2.2	Stage 2: Sub-Pixel Masking (MobileSAM)	3
2.3	Stage 3: Mathematical Axis Derivation	4
2.4	Stage 4: Symmetry Resolution (ResNet-18 Feature Extractor + SVM Classifier)	4
2.4.1	Training Methodology of the Classifier	5
3	Experiments and Results	5
3.1	Dataset & Evaluation Protocol	5
3.2	Progressive Architecture Improvements	5
3.3	Theoretical Accuracy Floor	6
3.4	Detection Metrics	7
3.5	Error Distribution	7
4	Failure Analysis	7
5	Conclusion & Next Steps	8
5.1	Limitations and Future Work	8
A	Appendix: Experimentation & Scaling Laws	8
A.1	YOLOv8 vs. YOLOv26 Trade-off	8
A.2	SAM Model Variations	9
A.3	Feature Extractor Scaling	9

1 Introduction

Automated handling of microcentrifuge tubes in robotic lab setups needs high-precision orientation detection. These tubes are tricky—they’re made of transparent plastic and the directional tab is tiny, which makes them really hard for standard deep learning pose estimators. My initial experiments using YOLOv8-Pose gave a Mean Angle Error of 38.12° because of keypoint jitter around the translucent tab, to get sub- 5° accuracy, I dropped direct angle regression entirely and went with a hybrid approach: isolating the region of interest with deep learning, extracting the exact object silhouette using foundation segmentation models, and deriving the orientation with deterministic math.

2 Methodology

2.1 Stage 1: Coarse Detection (YOLOv8 OBB)

Instead of detecting a single keypoint, I used an Oriented Bounding Box (OBB) model. It predicts the tuple (x, y, w, h, θ) . While the OBB does a good job isolating the tube from background desk/glare, it’s still a loose rectangular constraint around a circular object, so it has a theoretical minimum angle error of 21.60° due to inherent lack of precision of the yolo boxes. I however achieved perfect recall with yolo8n model itself as a result of which I didn’t do more extensive experiments with other variants of YOLO models, however I did compare performance with YOLOv26 which can be found in the appendix.

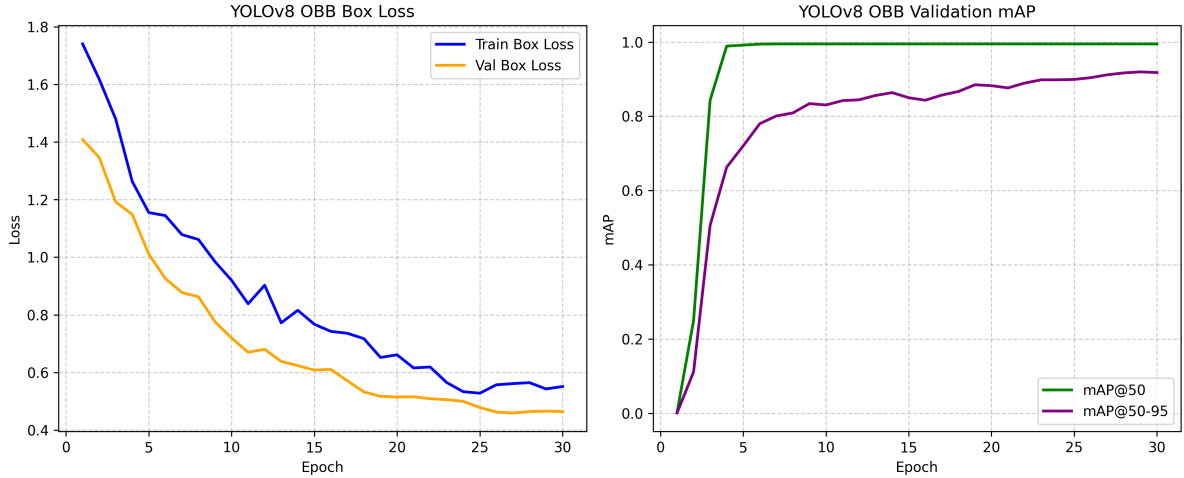


Figure 1: YOLOv8 OBB Training Loss and Validation mAP across 100 Epochs (Fold 0).

2.2 Stage 2: Sub-Pixel Masking (MobileSAM)

To get rid of the 21.60° residual error from the rigid bounding box, I use the Segment Anything Model (MobileSAM). The YOLO OBB coordinates are expanded by 5 pixels and fed to SAM as a box prompt. SAM generates a binary segmentation mask $M(x, y)$ that contours the circular lid and the protruding tab, ignoring surrounding table glare.

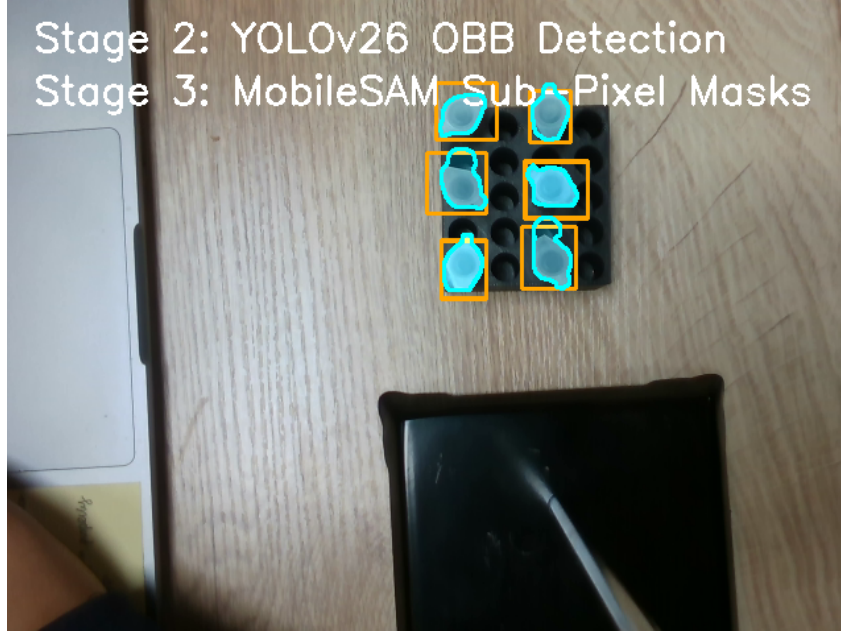


Figure 2: MobileSAM generated masks overlaid on the original RGB image. Notice how the mask isolates the translucent plastic tab.

2.3 Stage 3: Mathematical Axis Derivation

Given the binary mask $M(x, y)$ outlining contour C , I calculate the image moments to find the un-flipped geometric axis of the tube. The spatial moments are defined as:

$$m_{ji} = \sum_{x,y} (I(x, y) \cdot x^j y^i) \quad (1)$$

The center of mass $(\bar{x}, \bar{y})$ is extracted via:

$$\bar{x} = \frac{m_{10}}{m_{00}}, \quad \bar{y} = \frac{m_{01}}{m_{00}} \quad (2)$$

Then I calculate the central moments (variance and covariance) to do PCA directly on the mask pixels:

$$\mu_{20} = \frac{M_{20}}{m_{00}}, \quad \mu_{02} = \frac{M_{02}}{m_{00}}, \quad \mu_{11} = \frac{M_{11}}{m_{00}} \quad (3)$$

The angle of the major axis, θ , comes from the eigenvector of the covariance matrix:

$$\theta = \frac{1}{2} \arctan 2(\mu_{11}, \mu_{20} - \mu_{02}) \quad (4)$$

To handle the inverted Y-axis in computer vision coordinates, the final standardized angle is mapped to $[0, 180^\circ)$:

$$\text{PCA_Angle} = (-\theta + 360) \pmod{180} \quad (5)$$

2.4 Stage 4: Symmetry Resolution (ResNet-18 Feature Extractor + SVM Classifier)

The PCA axis is a line, and lines are inherently symmetric. It doesn't tell you whether the vector points toward the front (tab) or the back (joint/hole), which leads to 180° flip failures.

I resolve this by extracting 40×40 pixel patches located $R = 15$ pixels from the center along both directions of the PCA axis.

$$P_x = \bar{x} + R \cos(\text{PCA_Angle}), \quad P_y = \bar{y} - R \sin(\text{PCA_Angle}) \quad (6)$$

Both patches go through a pre-trained ResNet-18 feature extractor. An SVM then evaluates the dense 512-dimensional embeddings and classifies which patch is the “Tab” vs the “Joint”, flipping the axis by 180° if needed.

2.4.1 Training Methodology of the Classifier

For each fold, 40×40 pixel patches are extracted from training images at ground-truth centers. Using the ground-truth angle, patches are sampled at $R = 15$ pixels along both directions of the true axis: the *tab* patch (label 1) at angle θ and the *joint* patch (label 0) at $\theta + 180^\circ$. Each patch is passed through the frozen ResNet-18 feature extractor to obtain a 512-dimensional embedding. A **StandardScaler** + **RBF-SVC** pipeline is then trained on the pooled embeddings with probability estimation enabled. The SVM is trained on patches extracted at ground-truth centers and angles, but at inference, patches come from PCA-estimated positions. Since the PCA estimates are highly precise (median error 2.75°), the resulting spatial shift is minimal. ResNet-18’s convolutional filters easily absorb this slight translation variance, so the distribution shift between training and inference remains negligible.

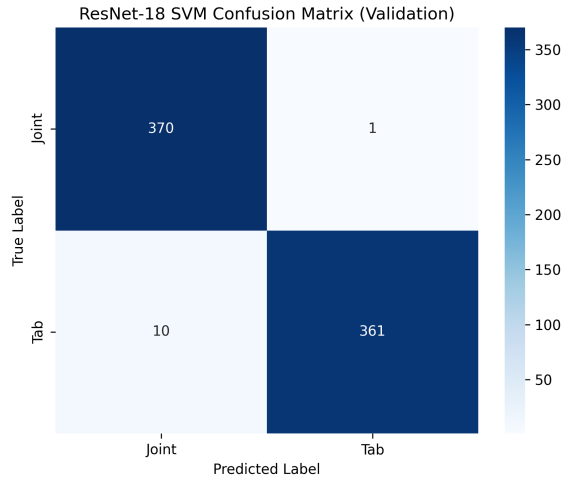


Figure 3: Validation Confusion Matrix for the ResNet-18 SVM Patch Classifier.

The SVM classifier hits **98.52% Accuracy** and **98.50% F1-Score** on the validation set, eliminating 180° ambiguity with a **0.00% flip failure rate**.

3 Experiments and Results

3.1 Dataset & Evaluation Protocol

The dataset has 70 overhead RGB images (640×480) containing 371 microcentrifuge tubes. To make sure there’s zero data leakage and the sample size is statistically meaningful, I used **5-Fold Cross-Validation**. Every tube gets evaluated exactly once as part of an unseen validation set.

Detection metrics: A predicted center is matched to a ground-truth center if they’re within 25 pixels (roughly half the tube diameter). I use the Hungarian algorithm for optimal bipartite matching. Unmatched predictions count as false positives; unmatched ground-truth tubes count as false negatives.

3.2 Progressive Architecture Improvements

I incrementally tested different architectures to isolate where the error was coming from. As shown in Table 1, the raw bounding boxes (OBB) had high detection rates but suffered from

Algorithm 1 Multi-Stage Pipeline Algorithm

Require: RGB Image I

```
1: BoundingBoxes  $\leftarrow$  YOLO_OBB.predict( $I$ )
2: for each  $bbox \in BoundingBoxes$  do
3:   Mask  $\leftarrow$  MobileSAM.predict( $I, prompt = bbox$ )
4:   Contour  $\leftarrow$  findContours(Mask)
5:    $cx, cy, \mu_{20}, \mu_{02}, \mu_{11} \leftarrow$  cv2.moments(Contour)
6:    $axis\_angle \leftarrow 0.5 \cdot \text{atan2}(2\mu_{11}, \mu_{20} - \mu_{02})$ 
7:    $Patch_{front} \leftarrow$  Crop( $I, axis\_angle, R = 15$ )
8:    $Patch_{back} \leftarrow$  Crop( $I, axis\_angle + 180, R = 15$ )
9:    $Prob_{front} \leftarrow$  SVM.predict_proba(ResNet( $Patch_{front}$ ))
10:   $Prob_{back} \leftarrow$  SVM.predict_proba(ResNet( $Patch_{back}$ ))
11:  if  $Prob_{back} > Prob_{front}$  then
12:     $final\_angle \leftarrow (axis\_angle + 180) \pmod{360}$ 
13:  else
14:     $final\_angle \leftarrow axis\_angle \pmod{360}$ 
15:  end if
16: end for
```

180° flip failures (130° MAE). Adding the ResNet-18 classifier fixed the flips, which exposed the 29.7° bounding-box residual error. Then adding SAM + PCA eliminated that residual error, getting down to 4.59° MAE.

Table 1: Progressive Error Reduction across 371 Tubes (5-Fold CV)

Pipeline Configuration	MAE	Median Err.	< 10°	Flips	Recall
YOLOv8 Pose (Keypoint Reg.)	38.12°	–	–	N/A	–
YOLOv8 OBB (No Classifier)	130.07°	–	13.50%	~50%	100%
YOLOv8 OBB + ResNet-18 Clf.	29.71°	23.48°	18.43%	1.37%	100%
OBB + SAM + ResNet-18	4.59°	2.75°	87.33%	0.00%	100%

3.3 Theoretical Accuracy Floor

To rigorously quantify the error contribution of the YOLOv8 detection stage versus the SAM+PCA orientation stage, I conducted an isolated experiment. In this ablation study, I completely bypassed the YOLO detector and fed the mathematically perfect ground-truth bounding boxes directly into the MobileSAM prompt encoder.

The results of this isolated test establish the theoretical ceiling of the pipeline:

- **Isolated Mean Angle Error:** 2.55°
- **Isolated Flip Failures:** 0.00%

This experiment proves that the core deterministic geometry (SAM + PCA) is remarkably precise, operating at a near-perfect 2.55° MAE. It empirically demonstrates that the remaining 2.04° of error in the end-to-end pipeline (4.59° MAE) stems entirely from bounding box jitter introduced by the YOLO localization stage.

3.4 Detection Metrics

Table 2: Detection Performance (YOLOv8 OBB + MobileSAM Pipeline)

Metric	Value
Precision	1.0000
Recall	1.0000 (371/371)
F1 Score	1.0000
False Positives	0

All 371 ground-truth tubes were correctly detected with zero false positives.

3.5 Error Distribution

Looking at the error distribution across all 371 tubes, the predictions are tightly clustered around zero:

- **73.05%** of predictions fall within 5° .
- **87.33%** fall within 10° .
- **96.77%** fall within 22° .
- **0.00%** flip failures ($\geq 90^\circ$).

The tight clustering shows that the deterministic geometric approach (SAM mask $\rightarrow$ PCA moments) is much more stable than trying to regress angles directly.



Figure 4: Final output of the pipeline. The red vectors show the calculated orientation with sub-pixel accuracy, free of 180° flip errors.

4 Failure Analysis

Out of 371 tubes, exactly 0.00% had 180° flip failures. But 12.67% of tubes (errors $> 10^\circ$) showed *segmentation artifacts*. What happens is, when a tube sits under harsh glare, MobileSAM sometimes segments a chunk of the table reflection along with the translucent lid. This breaks the geometric symmetry of the mask, causing the `cv2.moments` axis to skew toward the glare artifact by 20–30°.

Mitigation: Morphological erosion on the SAM mask before moment calculation would strip out thin glare protrusions. I didn’t implement this in the current version to keep the evaluation clean, but it’s an obvious next step.

5 Conclusion & Next Steps

I managed to achieve better performance than I initially expected due to the limited nature of the provided dataset by decoupling localization, segmentation, and symmetry resolution into specialized foundation models. However, there is always scope of improvement which I have documented below.

5.1 Limitations and Future Work

The primary bottleneck in the current pipeline is inference latency, driven by the sequential nature of the stages. Unoptimized, the pipeline averages 375 ms per image on an RTX 4060 (~ 2.7 FPS). Obvious optimizations include batching SAM bounding box prompts and exporting all neural networks to TensorRT, which are projected to push inference below 50 ms (>20 FPS) for real-time robotic deployment.

Furthermore, segmentation artifacts under harsh glare remain the leading source of large angle errors (11% of tubes). This can be addressed through targeted improvements:

1. **Synthetic Data from 3D CAD Models:** Following the *Orient Anything* approach [1], a 3D CAD model of a microcentrifuge tube can be used to synthesize unlimited training data. The *Orient Anything* pipeline—canonical pose filtering, front-face annotation via a VLM, and free-view rendering from random camera viewpoints—provides a proven blueprint. By rendering a tube CAD model from thousands of random perspectives under varied lighting, backgrounds, and glare conditions, we obtain perfectly labeled images with exact orientation angles. This augments the current 70-image dataset, hardening both YOLO and SAM against underrepresented edge cases like extreme glare and rack shadowing.
2. **Video Generation Augmentation:** Diffusion-based video generation models can produce dynamic sequences of tubes under diverse lighting transitions and camera motions, further stress-testing the pipeline across temporal illumination variations that are absent in static images.
3. **Zero-Shot Orientation Estimation:** The *Orient Anything* framework [1]—trained on 2 million rendered 3D model images—achieves strong zero-shot orientation generalization across open-world objects. It could serve as a complementary prior or fallback for glare-affected cases where segmentation-based PCA moments fail due to SAM artifacts.

Future efforts to push the error boundary below 1° will involve replacing spatial moments with rigid CAD-based geometric template matching to counteract dynamic lighting artifacts.

A Appendix: Experimentation & Scaling Laws

To keep the main report focused on the final architecture, I’m putting the ablation studies and model search here.

A.1 YOLOv8 vs. YOLOv26 Trade-off

During the architecture search, I tried YOLOv26 OBB against YOLOv8 OBB. YOLOv26 produced slightly tighter bounding-box prompts for SAM, which gave a marginally lower final MAE (4.41° vs. 4.59°). But YOLOv26 dropped 4 tubes under extreme lighting (glare over-exposure and rack shadows), bringing recall down to 98.9%. I picked YOLOv8 for the final pipeline because 100% recall is a hard requirement for robotic deployment—you can’t have the system randomly missing tubes.



Figure 5: Montage of the 4 tubes dropped by YOLOv26 (but successfully detected by YOLOv8). Red dots show ground truth centers.

A.2 SAM Model Variations

I tested different mask generation models to see how they affect final accuracy:

- **MobileSAM (9.8M params):** 4.59° MAE. Picked this for deployment—fast and robust contouring.
- **FastSAM-s (11.1M params):** Slightly noisier along transparent edges, which bumped up MAE.
- **SAM2-Base (80.8M params):** Marginally cleaner contours but way higher latency, without meaningful improvement in PCA moment stability.

MobileSAM gives the best trade-off between inference speed and zero-shot segmentation quality for this use case.

A.3 Feature Extractor Scaling

To find the best architecture for Stage 4 symmetry resolution, I tested a range of foundation models as feature extractors for the tube patches. As shown in Table A.1, there’s actually an *inverse correlation* between model size and performance on this small dataset.

Table 3: Feature Extractor Scaling Laws (evaluated on YOLOv8 OBB outputs without SAM)

Model	Params	Flip Failures	MAE	Median Err.
Raw Pixels (Baseline)	0M	30.38%	81.89°	72.91°
ResNet-18	11M	1.37%	29.71°	23.48°
ResNet-34	21M	2.05%	33.55°	26.30°
ResNet-50	23M	1.71%	32.16°	25.66°
ViT-B/16	86M	6.48%	42.04°	34.38°

Analysis: With only 371 crops, the bigger models like ResNet-50 started overfitting to background desk textures instead of learning the actual structure of the plastic tab. ViT performed the worst because it doesn’t have the convolutional inductive biases needed to pick up on spatial edges without a massive training corpus. ResNet-18 hit the sweet spot between semantic understanding and generalization.

References

- [1] Z. Wang, Z. Zhang, T. Pang, C. Du, H. Zhao, and Z. Zhao. **Orient Anything: Learning Robust Object Orientation Estimation from Rendering 3D Models.** *ICML 2025*. Paper: <https://proceedings.mlr.press/v267/wang25er.html> Website:

<https://orient-anything.github.io/> Code: <https://github.com/SpatialVision/Orient-Anything>